KUBERNETES PRACTICAL ASSESSMENT

Topic 11: Service & Networking | Topic 12: Storage

50 Scenario-Based, Troubleshooting & Misconception Multiple-Choice Questions (Single Best Answer)

Exam Overview:

- **Total Questions:** 50 single-choice questions (choose the single best answer A, B, C, or D).
- **Focus Areas:** Real-world troubleshooting, CLI trace outputs (`kubectl describe`, event logs), YAML configuration traps, and common architectural misconceptions.
- **Answer Key:** Full answer key with detailed technical explanations is provided at the end of this document.

Part 1: Kubernetes Services (ClusterIP, NodePort, LoadBalancer, Headless)

1. **[Misconception – NodePort Traffic Distribution]** A company exposes a web application on an on-premises 4-node Kubernetes cluster using a `NodePort` Service on port `30080`. External users are instructed to connect to `http://192.168.1.101:30080` (the IP of Worker Node 1). During peak hours, Worker Node 1 becomes severely CPU-throttled and eventually crashes, even though Worker Nodes 2, 3, and 4 are mostly idle. What happens to external user requests, and why?

   A. Kubernetes automatically migrates the `192.168.1.101` IP to Worker Node 2 so users experience zero downtime.

   B. A `NodePort` Service only load-balances traffic across Pods via its underlying `ClusterIP`, NOT across Worker Node IPs; since clients connect directly to Node 1's IP, all requests fail when Node 1 goes down.

   C. `kube-proxy` on Worker Nodes 2, 3, and 4 intercepts the failed TCP handshake to Node 1 and redirects clients.

   D. `NodePort` only opens port `30080` on nodes that currently host at least one running Pod for that Service.

2. **[Troubleshooting – Connection Refused]** You deploy an API Pod listening on container port `3000` and expose it with the following Service manifest:

   ```
   spec:
   selector: { app: payment-api }
   ports:
   - protocol: TCP
   port: 80
   targetPort: 8080
   ```

   When you run `curl http://payment-api.default.svc.cluster.local` from a debug Pod, you immediately receive `curl: (7) Failed to connect to payment-api port 80: Connection refused`. Running `kubectl get pods` shows the Pod is `1/1 Running`. What is the root cause?

   A. A `ClusterIP` Service cannot route traffic from port `80` to a non-privileged port above `1024`.

   B. If `targetPort` is set to `8080`, `kube-proxy` forwards packets to port `8080` on the Pod IP, where nothing is listening because the application binds to port `3000`.

   C. The `containerPort` field in the Pod spec automatically overrides `targetPort` in the Service spec.

   D. CoreDNS requires a `Headless` Service whenever `port` and `targetPort` have different values.

3. **[Trace Analysis – Empty Endpoints]** A developer reports that their `backend-svc` Service is unreachable inside the cluster. You run `kubectl describe svc backend-svc` and see:

```
Selector:          app=backend,tier=api
Type:              ClusterIP
IP:                10.96.45.12
Port:              http  80/TCP
TargetPort:        8080/TCP
Endpoints:         <none>
```

You check `kubectl get pods --show-labels` and see three Pods in `1/1 Running` state with labels `app=backend,env=prod`. Why is `Endpoints` showing <none>?

A. Service selectors evaluate multiple labels using logical `OR`, which causes a label conflict.

B. Service selectors evaluate all specified labels using logical `AND`; since the Pods lack the label `tier=api`, zero Pods match the Service selector.

C. The `ClusterIP` `10.96.45.12` is outside the Pod network CIDR.

D. You must manually create an `Endpoints` object whenever a Service uses more than one label in its selector.

4. [Troubleshooting – LoadBalancer Stuck Pending] You recently bootstrapped a self-managed Kubernetes cluster on three local VMware virtual machines using `kubeadm`. You create a Service with `type: LoadBalancer`, and after 15 minutes `kubectl get svc` still shows:

```
NAME         TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)
web-public   LoadBalancer   10.104.22.19   <pending>     80:31452/TCP
```

Despite `EXTERNAL-IP` being <pending>, what is currently functional, and why is it stuck?

A. The Service is completely non-functional; neither `ClusterIP` nor `NodePort` is allocated until `EXTERNAL-IP` is assigned.

B. Both the `ClusterIP` (`10.104.22.19:80`) and the `NodePort` (<NodeIP>:31452) are fully functional, but `EXTERNAL-IP` stays <pending> because there is no Cloud Controller Manager or bare-metal load balancer implementation (like MetalLB/Cilium LB-IPAM) present.

C. The `kube-apiserver` is waiting for a `GatewayClass` to approve the external IP allocation.

D. Port 31452 is blocked by a default Kubernetes `NetworkPolicy`.

5. [Practical Scenario – Preserving Source IP] Your security team requires your Nginx Pods to log the real public IP address of external clients connecting through a Cloud `LoadBalancer` Service. Currently, Nginx logs only show the internal private IP addresses of your Kubernetes Worker Nodes. You change the Service configuration to `externalTrafficPolicy: Local`. What side effect must you be prepared for?

A. Traffic routed by the Load Balancer to a Worker Node that does NOT have a local Ready Pod for that Service will be dropped rather than forwarded to another node.

B. All inter-Pod communication inside the cluster using the Service's `ClusterIP` will stop working.

C. Kubernetes will automatically convert the Deployment into a `DaemonSet`.

D. `kube-proxy` will disable TLS encryption on all incoming packets.

6. [Architecture – Headless Service] You are deploying a 3-node Apache Kafka or Stateful database cluster using a `StatefulSet` named `db` and a Service named `db-headless` configured with `clusterIP: None` and selector `app: db`. When an application Pod queries CoreDNS for `db-headless.default.svc.cluster.local`, what does CoreDNS return?

A. A single virtual IP address that round-robins traffic via `iptables`.

B. A `CNAME` record pointing to the Kubernetes API server.

C. Multiple DNS `A` (or `AAAA`) records containing the individual Pod IP addresses of all Ready Pods matching `app: db`, plus individual DNS names like `db-0.db-headless.default.svc.cluster.local`.

D. An error `SERVFAIL` because a Service without a `ClusterIP` cannot be registered in CoreDNS.

7. **[Misconception – ExternalName Service]** A developer wants to map an internal Service named `legacy-oracle` to an external database server located at IP address `172.16.50.100` on port `1521`. They create the following Service:

```
spec:
type: ExternalName
externalName: "172.16.50.100"
ports: [{ port: 3306, targetPort: 1521 }]
```

Why does this approach fail to work as the developer expects?

   A. `ExternalName` works purely at the DNS level by returning a `CNAME` record (which requires a valid DNS hostname, not a raw IPv4 address) and performs no proxying or port translation.

   B. `ExternalName` Services only work if a `StorageClass` is attached to the Service.

   C. `ExternalName` requires `kube-proxy` to run in `ipvs` mode to rewrite port `3306` to `1521`.

   D. `ExternalName` can only point to domains ending in `.svc.cluster.local`.

8. **[Troubleshooting – Readiness Probe Impact]** A Deployment has 3 replica Pods backing a `ClusterIP` Service. One Pod experiences a temporary deadlock; its `livenessProbe` still succeeds, so the container does not restart, but its `readinessProbe` fails (`0/1 Ready`). How does the Service handle traffic to this Pod by default?

   A. The Service continues sending 33% of requests to the Pod until the `livenessProbe` fails and kills the container.

   B. The EndpointSlice controller marks the Pod endpoint as `ready: false`, and `kube-proxy` immediately stops routing new Service connections to that Pod.

   C. The Service deletes the Pod and creates a replacement Pod on a different node.

   D. The entire Service goes offline until all 3 replicas become `1/1 Ready`.

9. **[Best Practice – Zero-Downtime Port Migration]** Your team is updating a Deployment where v1 of the container listens on port `8080`, while v2 changes the listening port to `9090`. During a RollingUpdate, both v1 and v2 Pods run simultaneously behind the same Service. How should you configure the Pod and Service manifests so neither v1 nor v2 requests fail during the rollout?

   A. Define two separate `ClusterIP` Services and force clients to retry both IPs.

   B. Assign a named port (e.g., `name: http-web`) to `containerPort: 8080` in v1 and `containerPort: 9090` in v2, and set `targetPort: http-web` in the Service.

   C. Set `targetPort: "8080-9090"` as a port range in the Service spec.

   D. Use `NodePort` because it automatically scans open ports inside containers.

10. **[Troubleshooting – Cross-Namespace DNS]** A Pod in the `checkout` Namespace tries to send an HTTP request to `http://inventory-svc:8080/items`, which is a `ClusterIP` Service located in the `warehouse` Namespace. The request fails with `Could not resolve host: inventory-svc`. What is the simplest fix?

   A. Change `inventory-svc` from `ClusterIP` to `NodePort`, because `ClusterIP` Services cannot be accessed across Namespaces.

   B. Update the request URL to use the namespace-qualified DNS name `http://inventory-svc.warehouse:8080/items` (or `inventory-svc.warehouse.svc.cluster.local`).

   C. Duplicate the `inventory-svc` Service manifest inside the `checkout` Namespace with the same label selector.

   D. Restart CoreDNS with the `--cross-namespace` flag enabled.

11. **[Practical Scenario – Internal Traffic Policy]** To reduce cross-node network latency charges in a multi-zone cloud cluster, an engineer sets `internalTrafficPolicy: Local` on a `ClusterIP` Service called `auth-cache`. However, the `auth-cache` Deployment only has 2 replicas across a 5-node cluster. What happens when a client Pod running on Node 5 (which hosts no `auth-cache` Pod) calls `auth-cache`?

   A. `kube-proxy` automatically falls back to routing the request to Node 1 or Node 2.

B. The connection fails (drops/refuses) because `internalTrafficPolicy: Local` strictly restricts traffic to endpoints residing on the exact same node as the originating client Pod.

C. The Kubernetes scheduler immediately migrates the client Pod from Node 5 to Node 1.

D. `internalTrafficPolicy: Local` is ignored for `ClusterIP` Services and only applies to `LoadBalancer` Services.

12. [Misconception – Session Affinity] An application stores user shopping carts in local Pod memory rather than Redis. The engineer enables `sessionAffinity: ClientIP` on the `ClusterIP` Service behind an Nginx Ingress Controller. However, external users still complain that their shopping carts randomly disappear on refresh. Why did `sessionAffinity: ClientIP` on the Service fail to keep external users pinned to the same Pod?

A. `sessionAffinity: ClientIP` is only supported on Windows Worker Nodes.

B. Traffic reaching the Service comes from the Nginx Ingress Controller Pods (which proxy the requests) or goes straight from Ingress Controller to Pod IPs via EndpointSlices, bypassing the Service's `ClientIP` NAT table or seeing only the Ingress Pod's internal IP.

C. `ClientIP` affinity resets every 5 seconds and cannot be configured via `sessionAffinityConfig`.

D. `ClusterIP` Services do not support `sessionAffinity`.

Part 2: Ingress & IngressClasses

13. [Troubleshooting – Blank Ingress Address] You apply a valid `Ingress` manifest to your cluster. Five minutes later, you run `kubectl get ingress` and observe:

```
NAME           CLASS    HOSTS              ADDRESS   PORTS   AGE
shop-ingress   <none>   shop.example.com             80      5m
```

External requests to `shop.example.com` time out. Which of the following is the MOST likely explanation?

A. The backend Pods referenced by `shop-ingress` have not created a `PersistentVolumeClaim` yet.

B. Either no Ingress Controller is installed in the cluster, or `spec.ingressClassName` was omitted and no `IngressClass` in the cluster is marked as the default class.

C. An `Ingress` resource only populates the `ADDRESS` column after receiving at least 100 HTTP requests.

D. Port 443 must be enabled before Kubernetes assigns an `ADDRESS` to an Ingress.

14. [Configuration Trap – Default IngressClass] A platform engineer installs both the NGINX Ingress Controller and the Traefik Ingress Controller in the same cluster. By mistake, the engineer adds the annotation `ingressclass.kubernetes.io/is-default-class: "true"` to BOTH the `nginx` and `traefik` `IngressClass` objects. What happens when a developer submits a new `Ingress` without specifying `spec.ingressClassName` (in Kubernetes v1.25+)?

A. Kubernetes automatically load-balances 50% of the Ingress rules to NGINX and 50% to Traefik.

B. The built-in `IngressClass` admission controller rejects the creation of the new `Ingress` resource with an error because more than one default `IngressClass` exists.

C. Kubernetes picks whichever `IngressClass` was created first by creation timestamp.

D. Both controllers delete each other's `LoadBalancer` Services.

15. [Misconception – Ingress Path Matching] Consider an `Ingress` rule configured with `path: /api/v1` and `pathType: Exact`, routing to `api-svc:80`. Which of the following incoming request URLs will match this rule and be forwarded to `api-svc`?

A. Both `/api/v1` and `/api/v1/` (with trailing slash).

B. `/api/v1/users` and `/api/v1`.

C. Only `/api/v1` (exact case-sensitive match; `/api/v1/` with a trailing slash will NOT match).

D. Any URL path that contains the substring `api/v1`.

16. **[Troubleshooting – Ingress Fake Certificate]** You configure TLS on an `Ingress` in the `production` Namespace referencing `secretName: wildcard-tls`. When testing with `curl -v https://app.example.com`, TLS fails with:

```
SSL certificate problem: unable to get local issuer certificate
subject: O=Acme Co; CN=Kubernetes Ingress Controller Fake Certificate
```

You check `kubectl get secret wildcard-tls -n default` and confirm the certificate is valid. Why is the Ingress Controller serving the "Fake Certificate"?

A. An `Ingress` can only reference a TLS `Secret` that resides in the exact SAME Namespace (`production`) as the `Ingress` resource itself.

B. The `Secret` was created with `type: kubernetes.io/tls` instead of `type: Opaque`.

C. `ClusterIP` Services behind an Ingress must also mount the TLS `Secret`.

D. The Ingress Controller only reloads TLS certificates when Worker Nodes are rebooted.

17. **[Trace Analysis – HTTP 503 on Ingress]** Users visiting `https://portal.example.com/login` receive an HTTP `503 Service Temporarily Unavailable` response directly from NGINX Ingress. What does an HTTP `503` at the Ingress layer almost always indicate in Kubernetes?

A. The client's IP address is blocked by a `StorageClass` policy.

B. The Ingress Controller matched the Host and Path rule, but the target Service either does not exist or currently has zero Ready endpoints in its `EndpointSlice`.

C. The TLS certificate in the Ingress `Secret` has expired.

D. The `IngressClass` parameter reference is missing.

18. **[Misconception – Exposing TCP Databases via Ingress]** A developer writes a standard Kubernetes `Ingress` manifest (`apiVersion: networking.k8s.io/v1`) to expose a MySQL database on port `3306` using `host: db.example.com`. Why is this invalid in the standard Kubernetes `Ingress` API?

A. Standard `Ingress` resources only support Layer 7 HTTP and HTTPS routing rules; they do not natively define arbitrary Layer 4 TCP/UDP port listeners in the Ingress spec.

B. Port `3306` is reserved by `kubelet` on all nodes.

C. Standard `Ingress` only supports UDP protocols like DNS and QUIC.

D. MySQL requires an `ExternalName` Service to work with `Ingress`.

19. **[Architecture – IngressClass Controller Mapping]** You inspect an `IngressClass` manifest:

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
name: internal-nginx
spec:
controller: k8s.io/ingress-nginx-internal
```

How does an Ingress Controller Pod running in the cluster know that it is responsible for reconciling `Ingress` objects that specify `ingressClassName: internal-nginx`?

A. The controller matches its own Deployment name against `metadata.name` of the `IngressClass`.

B. The controller watches `IngressClass` objects whose `spec.controller` string matches the controller's configured controller-class identifier flag.

C. Kubernetes schedules the Ingress Controller Pod only on nodes labeled with `internal-nginx`.

D. CoreDNS forwards `IngressClass` events to the controller over port 53.

20. **[Practical Scenario – Ingress Default Backend]** An `Ingress` object defines `spec.defaultBackend` pointing to `custom-404-svc:8080`, alongside rules for `host: api.example.com`. When will traffic be routed to `custom-404-svc:8080`?

A. Whenever a Pod inside `api.example.com` returns an HTTP 500 Internal Server Error.

B. Whenever an incoming request to the Ingress Controller does not match any configured `host` or `path` rule in the `Ingress` spec.

C. Whenever the cluster's CoreDNS service goes down.

D. Only when the client sends a `DELETE` HTTP request.

Part 3: Gateway API

21. [Architecture – Role-Oriented Design] Gateway API splits routing configuration across three primary resource kinds to match organizational personas. Which mapping is ACCURATE according to official Kubernetes Gateway API documentation?

A. `GatewayClass` → Application Developer; `Gateway` → Infrastructure Provider; `HTTPRoute` → Cluster Operator.

B. `GatewayClass` → Infrastructure Provider; `Gateway` → Cluster Operator; `HTTPRoute` → Application Developer.

C. `GatewayClass` → End User; `Gateway` → Application Developer; `HTTPRoute` → Security Auditor.

D. All three resources must be created in the `kube-system` Namespace by the Cluster Admin.

22. [Troubleshooting – Cross-Namespace Backend Reference] An Application Developer creates an `HTTPRoute` in the `storefront` Namespace that routes traffic to a backend `Service` named `auth-svc` in the `security` Namespace. Traffic fails with HTTP 500, and `kubectl describe httproute -n storefront` shows:

```
Conditions:
Type:    ResolvedRefs
Status:  False
Reason:  RefNotPermitted
```

What resource must be created, and in which Namespace, to resolve `RefNotPermitted`?

A. A `ReferenceGrant` resource in the `security` (target) Namespace explicitly permitting `HTTPRoute` objects from the `storefront` Namespace to reference `Service` resources.

B. A `ReferenceGrant` resource in the `storefront` (source) Namespace.

C. A `NetworkPolicy` in `kube-system`.

D. An `ExternalName` Service in the `security` Namespace.

23. [Troubleshooting – Route Attachment Rejected] A Cluster Operator creates a shared `Gateway` named `corp-gw` in the `infra-gw` Namespace using default listener settings:

```
listeners:
- name: http
protocol: HTTP
port: 80
```

A developer creates an `HTTPRoute` in the `billing` Namespace referencing `parentRefs: [{ name: corp-gw, namespace: infra-gw }]`. Running `kubectl describe httproute -n billing` shows `Accepted: False` with `Reason: NotAllowedByListeners`. Why?

A. By default, a Gateway listener sets `allowedRoutes.namespaces.from: Same`, which only allows Routes located in the same Namespace (`infra-gw`) to attach unless changed to `All` or `Selector`.

B. `HTTPRoute` can only attach to HTTPS listeners on port 443.

C. The `billing` Namespace does not have a `GatewayClass` installed inside it.

D. A `Gateway` can only accept a maximum of one `HTTPRoute` across the entire cluster.

24. [Practical Scenario – Canary Traffic Splitting] You want to perform a Canary release where 95% of HTTP requests go to `orders-v1` Service and 5% go to `orders-v2` Service using Gateway API. How is this configured in `HTTPRoute`?

A. By adding vendor-specific NGINX annotations to the HTTPRoute metadata.

B. By listing both `orders-v1` (`weight: 95`) and `orders-v2` (`weight: 5`) under a single rule's `backendRefs` array natively in the HTTPRoute spec.

C. By creating 19 Pod replicas for `orders-v1` and 1 Pod replica for `orders-v2` behind a single Service.

D. By creating two separate `GatewayClass` objects with different weights.

25. [Troubleshooting – Gateway Not Programmed] You apply a `Gateway` manifest referencing `gatewayClassName: istio-gw`. When checking `kubectl get gateway`, the status shows `PROGRAMMED: False` and `Accepted: False` (`Waiting for controller`). What is the root cause?

A. Gateway API resources are declarative; without a running Gateway controller in the cluster that implements the `istio-gw` `GatewayClass` (and without the Gateway API CRDs reconciled by that controller), no data-plane proxy is provisioned.

B. You must create a `NodePort` Service manually before creating a `Gateway`.

C. `Gateway` resources only work when `kube-proxy` is disabled.

D. The `Gateway` name must always be `default`.

26. [Feature Comparison – Gateway API vs Ingress] Which of the following features is natively standardized inside the core Kubernetes Gateway API specification (`HTTPRoute`) without requiring custom vendor annotations?

A. Request Header modification (`RequestHeaderModifier`), Request Mirroring (`RequestMirror`), URL Redirects/Rewrites, and weighted traffic splitting.

B. Automatic creation of `PersistentVolumes` for stateful databases.

C. Container image vulnerability scanning during Pod startup.

D. Replacing `etcd` with an external SQL database.

27. [Configuration Logic – Hostname Intersection] A `Gateway` listener is configured with `hostname: "*.shop.example.com"`. An `HTTPRoute` attaches to this listener and specifies `hostnames: ["api.shop.example.com", "admin.otherdomain.org"]`. For which hostname(s) will traffic actually be routed by this `HTTPRoute` through this `Gateway`?

A. Both `api.shop.example.com` and `admin.otherdomain.org`.

B. Only `api.shop.example.com`, because Gateway API computes the intersection between the Listener's hostname and the Route's hostnames.

C. Neither hostname, because wildcard listeners reject all specific route hostnames.

D. Only `admin.otherdomain.org`.

Part 4: Network Policies & EndpointSlices

28. [YAML Trap – AND vs OR in NetworkPolicy] A security engineer wants to allow ingress traffic to db Pods ONLY from Pods that are BOTH in a Namespace labeled `env: prod` AND carry the Pod label `role: backend`. However, they accidentally write the following `ingress.from` block with two list hyphens (`-`):

```
ingress:
- from:
- namespaceSelector:
matchLabels: { env: prod }
- podSelector:
matchLabels: { role: backend }
```

What did this configuration actually allow?

A. It blocks all traffic because `namespaceSelector` and `podSelector` cannot be used in the same policy.

B. Because of the second `-`, the two selectors are evaluated as logical OR: it allows traffic from ANY Pod inside the `env: prod` Namespace, OR from ANY Pod labeled `role: backend` in the policy's own Namespace!

C. It strictly enforces the logical AND condition across all Namespaces as intended.

D. It allows traffic from external internet IPs.

29. [Troubleshooting – Broken DNS After Egress Lock] You apply a strict `NetworkPolicy` to a microservice Pod in the `payment` Namespace:

```
spec:
podSelector: { matchLabels: { app: stripe-worker } }
policyTypes: [Ingress, Egress]
egress:
- to:
- ipBlock: { cidr: 0.0.0.0/0 }
ports:
- protocol: TCP
port: 443
```

Immediately after applying this policy, `stripe-worker` logs `curl: (6) Could not resolve host: api.stripe.com`. However, `curl https://1.1.1.1` works fine. Why is hostname resolution failing?

A. The Pod is isolated for `Egress` and only TCP port 443 is allowed; outbound DNS queries on UDP/TCP port 53 to CoreDNS (in `kube-system`) are being blocked by the policy.

B. `ipBlock: 0.0.0.0/0` automatically blocks all traffic to internal `ClusterIP` Services.

C. CoreDNS requires TCP port 80 to be open on all application Pods.

D. `NetworkPolicy` disables the Pod's `/etc/resolv.conf` file.

30. [Misconception – NetworkPolicy Without CNI Support] You spin up a test cluster using a basic CNI plugin (such as default Flannel without Calico/Cilium policy engine). You apply a `default-deny-all` `NetworkPolicy` in the `default` Namespace. `kubectl get networkpolicy` shows the policy was created without any error. When you test connectivity between Pods, what happens?

A. All traffic is blocked by `kube-proxy` using `iptables`.

B. The Kubernetes API server stores the `NetworkPolicy` object, but since the underlying CNI plugin does not implement a NetworkPolicy controller, the policy is silently ignored and all Pods can still communicate freely.

C. All Pods in the `default` Namespace are prevented from starting until Calico is installed.

D. Only TCP traffic is blocked while UDP traffic remains open.

31. [Architecture – Directional Isolation] A Pod labeled `app: redis` has NO `NetworkPolicy` selecting it initially. You then apply ONE `NetworkPolicy` that selects `app: redis` with `policyTypes: ["Ingress"]` and allows TCP port 6379 from `app: api`. No `egress` rules or `Egress` policyType are mentioned. If someone execs into the `redis` Pod and runs `curl https://example.com` (outbound connection), what happens?

A. The outbound connection is blocked because selecting a Pod in any `NetworkPolicy` isolates both Ingress and Egress simultaneously.

B. The outbound connection succeeds because isolation is evaluated independently per direction; since no policy applies `policyTypes: ["Egress"]` to `app: redis`, its egress remains completely non-isolated (open).

C. The outbound connection is only allowed to `app: api` Pods on port 6379.

D. Redis crashes with a socket permission error.

32. [Practical Scenario – Empty podSelector Default Deny] What is the effect of applying the following `NetworkPolicy` in the `finance` Namespace?

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: deny-all-ingress
namespace: finance
spec:
podSelector: {}
policyTypes:
- Ingress
```

A. Because `podSelector: {}` is empty, it selects NO Pods and has zero effect on the Namespace.

B. Because `podSelector: {}` matches ALL Pods in the `finance` Namespace and specifies no `ingress` allow rules, it blocks all incoming traffic to every Pod in the `finance` Namespace (while leaving outbound Egress traffic untouched).

C. It deletes all Services in the `finance` Namespace.

D. It blocks incoming traffic across every Namespace in the entire cluster.

33. [Architecture – Why EndpointSlices Replaced Endpoints] In a large production cluster, a single `Service` fronts 2,500 Pods. Under the legacy monolithic `Endpoints` API, why did rolling updates of this Deployment cause severe control-plane CPU spikes and network saturation across all 500 Worker Nodes?

A. Every time a single Pod started or terminated, the entire monolithic `Endpoints` object (containing all 2,500 IP addresses) had to be serialized by `kube-apiserver` and transmitted in full to `kube-proxy` on all 500 nodes.

B. Legacy `Endpoints` required each Pod to hold an exclusive lock on `etcd`.

C. Legacy `Endpoints` could only store a maximum of 50 IP addresses before crashing `CoreDNS`.

D. `Endpoints` did not support TCP connections.

34. [Configuration & Scaling – EndpointSlice Limits] By default, how many endpoints does Kubernetes pack into a single `EndpointSlice` resource before creating an additional `EndpointSlice` for the same Service?

A. 10 endpoints per slice (hardcoded).

B. 100 endpoints per slice by default (configurable up to 1,000 via `--max-endpoints-per-slice` on `kube-controller-manager`).

C. 1 endpoint per slice (1-to-1 mapping with Pods).

D. 65,535 endpoints per slice.

35. [Troubleshooting – Graceful Termination & EndpointSlice Conditions] Each endpoint inside an `EndpointSlice` tracks three boolean conditions: `ready`, `serving`, and `terminating`. What is the difference between `ready` and `serving` when a Pod is being gracefully terminated (`deletionTimestamp` is set)?

A. `ready` is immediately set to `false` as soon as a Pod enters `Terminating`, whereas `serving` remains `true` as long as the terminating Pod is still passing its readiness probe, allowing proxies to drain or fail-open gracefully if all other Pods are down.

B. `serving` is only used for `StorageClass` provisioners.

C. Both `ready` and `serving` are always identical in all states.

D. `terminating` is set to `false` when a Pod is being deleted.

Part 5: Kubernetes Storage (PV, PVC, StorageClasses)

36. [Trace Analysis – WaitForFirstConsumer] A developer creates a `PersistentVolumeClaim` named `app-data-pvc` using the `gp3-topology` `StorageClass`. Running `kubectl describe pvc app-data-pvc` shows Status Pending with the following event:

```
Normal  WaitForFirstConsumer  persistentvolume-controller
waiting for first consumer to be created before binding
```

No Pod has been created yet. What should the developer do to make the PVC transition to Bound?

A. Delete and recreate the `StorageClass` because the CSI driver has crashed.

B. Deploy a Pod that mounts `app-data-pvc`; once the scheduler picks a Node for that Pod, the CSI provisioner will dynamically create the PV in that Node's topology/zone and bind the PVC.

C. Manually create a static `PersistentVolume` in the `kube-system` Namespace.

D. Restart `kube-proxy` on all Worker Nodes.

37. [Troubleshooting – Multi-AZ Volume Node Affinity Conflict] Your AWS EKS cluster spans three Availability Zones (`us-east-1a`, `us-east-1b`, `us-east-1c`). A `StorageClass` for AWS EBS is configured with `volumeBindingMode: Immediate`. A PVC is created first, and AWS dynamically provisions an EBS volume in `us-east-1a`. Later, a Deployment is created that mounts this PVC, but uses a `nodeSelector` forcing the Pod onto a GPU node that only exists in `us-east-1c`. What error will `kubectl describe pod` show?

A. `0/6 nodes are available: 2 node(s) didn't match Pod's node affinity/selector, 4 node(s) had volume node affinity conflict.`

B. `CrashLoopBackOff: Back-off restarting failed container.`

C. `ErrImagePull: failed to pull EBS volume image.`

D. AWS EBS automatically migrates the disk across Availability Zones from 1a to 1c within 5 seconds.

38. [Troubleshooting – PV Stuck in Released State] A database `PersistentVolume` (`pv-mysql-prod`) was configured with `persistentVolumeReclaimPolicy: Retain`. An engineer deletes the `mysql-pvc` claim to recreate it with updated labels. However, when the new `mysql-pvc` is created, it stays stuck in `Pending`, while `kubectl get pv` shows:

```
NAME            CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS
CLAIM
pv-mysql-prod   100Gi      RWO            Retain           Released
default/mysql-pvc
```

Why won't the new `mysql-pvc` bind to `pv-mysql-prod`, and how do you make `pv-mysql-prod` `Available` again without losing the existing data on the disk?

A. A `Released` PV still holds the old PVC's `spec.claimRef` UID (since previous data remains on the volume); an admin must edit the PV and remove the `spec.claimRef` block so its status returns to `Available`.

B. You must format the underlying physical disk before Kubernetes allows a PV to change from `Released` to `Available`.

C. Change `persistentVolumeReclaimPolicy` from `Retain` to `Delete` so it binds to the new PVC.

D. Rename the Worker Node where the PV was mounted.

39. [Misconception – ReadWriteOnce (RWO) vs ReadWriteOncePod (RWOP)] A `PersistentVolumeClaim` is bound to an AWS EBS volume with `accessModes: ["ReadWriteOnce"]`. Two different Pods (`pod-writer-a` and `pod-reader-b`) are scheduled onto the **exact same Worker Node** (`node-01`) and both reference this same PVC. What happens when both Pods start on `node-01`?

A. The second Pod fails with a `Multi-Attach error` because `ReadWriteOnce` only allows one Pod in the cluster to mount the volume.

B. Both Pods successfully mount and access the volume simultaneously because `ReadWriteOnce` (`RWO`) restricts mounting to a **single Node**, NOT a single Pod (whereas `ReadWriteOncePod` / `RWOP` restricts mounting to a single Pod).

C. Kubernetes deletes the first Pod automatically.

D. The volume is converted into `ReadOnlyMany` (`ROX`) automatically.

40. [Trace Analysis – Multi-Attach Error] You have a stateless web Deployment that someone modified to mount a single AWS EBS PVC (`accessModes: [ReadWriteOnce]`). When you scale the Deployment from 1 replica to 3 replicas across 3 different Worker Nodes, 2 of the Pods remain stuck in `ContainerCreating`. Running `kubectl describe pod` on one of the stuck Pods shows:

```
Warning  FailedAttachVolume  attachdetach-controller
Multi-Attach error for volume "pvc-8f92a..." Volume is already
exclusively attached to one node and can't be attached to another
```

Which architectural change properly solves this if all 3 Pods on different nodes genuinely need shared read/write access to the same files?

- **A.** Migrate to a shared filesystem storage backend that supports `ReadWriteMany` (`RWX`), such as NFS, AWS EFS, Azure Files, or CephFS.
- **B.** Change the EBS PVC manifest string from `ReadWriteOnce` to `ReadWriteMany` without changing the underlying EBS storage type.
- **C.** Add `allowVolumeExpansion: true` to the EBS `StorageClass`.
- **D.** Change the Service type from `ClusterIP` to `LoadBalancer`.

41. [Capacity Binding Behavior] In a cluster with no dynamic provisioning (`storageClassName: ""`), an administrator has pre-created only one `Available` static `PersistentVolume` of size `50Gi` with `RWO`. A developer creates a `PersistentVolumeClaim` requesting `storage: 10Gi` with `RWO`. What happens?

- **A.** The PVC stays `Pending` forever because `10Gi` does not exactly equal `50Gi`.
- **B.** The `10Gi` PVC binds exclusively (1-to-1) to the `50Gi` PV; the Pod gets access to the entire `50Gi` volume, and the remaining `40Gi` cannot be claimed by any other PVC.
- **C.** Kubernetes splits the `50Gi` PV into one `10Gi` PV and one `40Gi` PV automatically.
- **D.** The PVC creation is rejected by the admission webhook.

42. [Troubleshooting – Static PV Ignored Due to Default StorageClass] An admin manually creates a static `PersistentVolume` with `capacity: 20Gi`, `accessModes: [ReadWriteOnce]`, and NO `storageClassName` field set. A developer creates a matching `PersistentVolumeClaim` requesting `20Gi` and `ReadWriteOnce`, also omitting `storageClassName` from the PVC YAML. Surprisingly, instead of binding to the admin's static PV, Kubernetes dynamically provisions a brand-new cloud disk! Why?

- **A.** When `storageClassName` is omitted from a PVC (not set at all) and the cluster has a default `StorageClass` enabled, the `DefaultStorageClass` admission controller automatically populates `spec.storageClassName` on the PVC with the default class; to bind to a PV with no class, the PVC must explicitly set `storageClassName: ""`.
- **B.** Static PVs can only be bound by `StatefulSets`, never by standalone PVCs.
- **C.** A PVC always ignores static PVs if their capacity is greater than `10Gi`.
- **D.** The static PV was created in the wrong Namespace.

43. [Troubleshooting – PVC Stuck in Terminating] You run `kubectl delete pvc data-volume-claim`. The command hangs, and in another terminal `kubectl get pvc` shows `data-volume-claim` has been in `Terminating` status for 10 minutes. You inspect the PVC YAML and see `finalizers: [kubernetes.io/pvc-protection]`. Why is Kubernetes refusing to finalize the deletion of this PVC?

- **A.** The Storage Protection feature keeps the `kubernetes.io/pvc-protection` finalizer active as long as at least one active Pod is still scheduling or running with that PVC mounted; once the Pod is deleted, the PVC is immediately removed.
- **B.** `etcd` has lost quorum.
- **C.** A PVC with `ReadWriteOnce` can only be deleted by the root user on the Worker Node.
- **D.** You must delete the `StorageClass` before you can delete a PVC.

44. [Practical Scenario – Online Volume Expansion] A production PostgreSQL disk is 92% full. The engineer edits the `PersistentVolumeClaim` and changes `spec.resources.requests.storage` from `50Gi` to `100Gi`, but `kubectl apply` returns:

```
Forbidden: only dynamically provisioned pvc can be resized and the
storageclass that provisions the pvc must support resize
```

Which setting is missing on the StorageClass?

A. `allowVolumeExpansion: true`

B. `volumeBindingMode: Immediate`

C. `reclaimPolicy: Retain`

D. `mountOptions: ["resize"]`

45. [Trace Analysis – FileSystemResizePending] After enabling `allowVolumeExpansion: true` on the StorageClass, you increase a PVC's size from `20Gi` to `50Gi`. Checking `kubectl describe pvc` shows the condition `FileSystemResizePending: True` with message:

```
Waiting for user to (re-)start a pod to finish file system resize of
volume on node.
```

What has already completed, and what is still pending?

A. The underlying cloud/storage controller has already expanded the physical block volume to `50Gi`, and kubelet/CSI Node plugin now needs a Pod mount (or Pod restart if online expansion isn't supported for that state) to expand the Linux filesystem (`ext4`/`xfs`) inside the volume.

B. The cloud provider rejected the resize due to insufficient quota.

C. You must delete the PVC and recreate it to apply the `50Gi` filesystem change.

D. The `PersistentVolume` was deleted by the CSI driver.

46. [Misconception – Shrinking a PVC] An engineer accidentally requested `500Gi` instead of `50Gi` in a `PersistentVolumeClaim` that is now Bound. They edit the PVC YAML to change `spec.resources.requests.storage` from `500Gi` down to `50Gi` and run `kubectl apply`. What happens?

A. The API server rejects the update with `Forbidden: field can not be less than previous value`, because Kubernetes does not support shrinking existing PVC storage sizes in-place.

B. Kubernetes shrinks the volume to `50Gi` and truncates the last `450Gi` of blocks.

C. Kubernetes automatically creates a new `50Gi` PVC and copies all files over via `rsync`.

D. The Pod is restarted in read-only mode.

47. [Production Incident – Accidental Data Deletion] A cluster uses a custom `StorageClass` named `fast-ssd` where the `reclaimPolicy` field was omitted when the `StorageClass` was created. A developer deletes a `PersistentVolumeClaim` provisioned by `fast-ssd`, assuming the PV data will be preserved. To their horror, both the `PersistentVolume` object and the underlying AWS EBS disk are permanently deleted! Why?

A. For PVs dynamically provisioned by a `StorageClass`, if `reclaimPolicy` is not explicitly specified in the `StorageClass`, it defaults to `Delete` (not `Retain`).

B. AWS EBS deletes all volumes that are unmounted for more than 10 seconds.

C. Deleting a PVC always formats the disk even when `reclaimPolicy` is `Retain`.

D. The `kubernetes.io/pvc-protection` finalizer deletes underlying disks upon PVC removal.

48. [Troubleshooting – Raw Block Volume Mode] A high-performance database requires direct access to a raw block device without any filesystem overhead, so the PVC and PV are configured with `volumeMode: Block`. However, when the developer writes the Pod manifest using standard `containers[].volumeMounts`, the Pod fails validation/startup. How must a Pod mount a `volumeMode: Block` PVC?

A. Using `containers[].volumeDevices` with a `devicePath` (e.g., `/dev/xvda`) instead of `volumeMounts` with a `mountPath`.

B. By running an `initContainer` that mounts the PVC over NFS.

C. By setting `hostNetwork: true` on the Pod.

D. Kubernetes Pods cannot use `volumeMode: Block`.

49. [Practical Scenario – StatefulSet Scale Down & PVC Lifecycle] You have a 3-replica `StatefulSet` named `redis-cluster` that uses `volumeClaimTemplates` to dynamically create `data-redis-cluster-0`, `data-redis-cluster-1`, and `data-redis-cluster-2`. You run `kubectl scale statefulset redis-cluster --replicas=1`. Pod `redis-cluster-2` and `redis-cluster-1` are terminated. What happens to their PVCs (`data-redis-cluster-1` and `data-redis-cluster-2`) by default?

A. Both PVCs and their underlying PVs are immediately deleted as soon as the Pods terminate.

B. Both PVCs remain intact and Bound in the Namespace to protect stateful data against accidental scale-down; if you later scale back up to 3 replicas, the new Pods automatically reattach to their existing PVCs.

C. Their data is merged into `data-redis-cluster-0`.

D. The PVCs are moved to the `default` Namespace.

50. [Troubleshooting – Non-Root Container Permission Denied on Mounted PV] A security-hardened Pod runs its container as non-root user `uid=10001` (`runAsNonRoot: true`). When the Pod mounts a newly provisioned ext4 `PersistentVolumeClaim` at `/var/lib/app/data`, the application crashes on startup with `Permission denied: cannot create /var/lib/app/data/lockfile` because the root directory of a fresh ext4 filesystem is owned by `root:root` (`0755`). What is the cleanest Kubernetes-native Pod setting to fix this without running a privileged root container?

A. Set `spec.securityContext.fsGroup: 10001` at the Pod level so `kubelet` automatically changes group ownership and permissions of the volume's files to GID `10001` when mounting the volume.

B. Change the `StorageClass` `volumeBindingMode` to `WaitForFirstConsumer`.

C. Change the PVC access mode to `ReadOnlyMany`.

D. Expose the Pod with a `LoadBalancer` Service.

Answer Key & Detailed Practical Explanations

Quick Reference Answer Grid

Q1 – Q10	Q11 – Q20	Q21 – Q30	Q31 – Q40	Q41 – Q50
1. **B**	11. **B**	21. **B**	31. **B**	41. **B**
2. **B**	12. **B**	22. **A**	32. **B**	42. **A**
3. **B**	13. **B**	23. **A**	33. **A**	43. **A**
4. **B**	14. **B**	24. **B**	34. **B**	44. **A**
5. **A**	15. **C**	25. **A**	35. **A**	45. **A**
6. **C**	16. **A**	26. **A**	36. **B**	46. **A**
7. **A**	17. **B**	27. **B**	37. **A**	47. **A**
8. **B**	18. **A**	28. **B**	38. **A**	48. **A**
9. **B**	19. **B**	29. **A**	39. **B**	49. **B**
10. **B**	20. **B**	30. **B**	40. **A**	50. **A**

Detailed Explanations

Q1. Answer: B. As emphasized in KodeKloud's Service architecture guide, `NodePort` builds on top of `ClusterIP`. While `ClusterIP` load-balances traffic across backend Pods, `NodePort` does **not** load-balance across Worker Nodes. Clients connecting directly to `Node1_IP:30080` hit only Node 1; if Node 1 fails, those requests fail. A `LoadBalancer` in front of the nodes is needed to distribute traffic across healthy nodes.

Q2. Answer: B. In a Service definition, `port` (`80`) is the port exposed on the Service's virtual `ClusterIP`, while `targetPort` (`8080`) is the destination port on the Pod where packets are sent. Since the container is actually listening on port `3000`, packets arriving at `PodIP:8080` receive a TCP `RST` (`Connection refused`).

Q3. Answer: B. Service label selectors use logical `AND` matching. A Pod must possess **every** label key-value pair listed in `spec.selector` (`app=backend` AND `tier=api`). Because the Pods only have `app=backend,env=prod`, no Pods match, leaving `Endpoints: <none>`.

Q4. Answer: B. A `LoadBalancer` Service is a superset of `NodePort` and `ClusterIP`. Even when `EXTERNAL-IP` is `<pending>` (because bare-metal clusters lack a Cloud Controller Manager or MetalLB), Kubernetes immediately allocates both the internal `ClusterIP` and the `NodePort` (31452).

Q5. Answer: A. Setting `externalTrafficPolicy: Local` instructs `kube-proxy` to only route traffic to Pods residing on the local node that received the packet, avoiding an extra cross-node SNAT hop and preserving the client's source IP. However, if a node has zero local Pods for that Service, incoming packets to that node are dropped (though cloud load balancers use the Service's `healthCheckNodePort` to stop sending traffic to such nodes).

Q6. Answer: C. A Headless Service (`clusterIP: None`) does not allocate a virtual IP. Instead, CoreDNS returns multiple `A`/`AAAA` records directly mapping to the ready Pod IPs, and when paired with a `StatefulSet`, creates deterministic DNS records for each Pod (`<pod-name>.<service-name>.<namespace>.svc.cluster.local`).

Q7. Answer: A. An `ExternalName` Service works exclusively at the DNS layer by returning a `CNAME` record pointing to a DNS domain name. It cannot point to a raw IPv4 address, and it does not proxy or translate ports (to route to an external IP, you create a selector-less Service with a manual `EndpointSlice`).

Q8. Answer: B. Readiness probes control whether a Pod receives traffic from Services. When a Pod transitions to `Ready: False`, the EndpointSlice controller marks its endpoint condition as `ready: false`, and `kube-proxy` removes it from active load balancing (unless `publishNotReadyAddresses: true` is explicitly set on the Service).

Q9. Answer: B. Kubernetes allows `targetPort` in a Service to reference a string **port name** defined in the Pod's `containers[].ports[].name`. During a rolling update, v1 Pods can map `http-web` →

8080 and v2 Pods can map http-web → 9090, allowing seamless traffic routing to both versions simultaneously.

Q10. Answer: B. Short Service names (like inventory-svc) only resolve within the same Namespace via the Pod's /etc/resolv.conf search domains. To reach a Service in another Namespace (warehouse), the client must append the Namespace (inventory-svc.warehouse or the full FQDN inventory-svc.warehouse.svc.cluster.local).

Q11. Answer: B. internalTrafficPolicy: Local forces internal cluster traffic sent to a Service ClusterIP to be routed exclusively to endpoints running on the same node as the client Pod. If no Ready endpoint exists on Node 5, the connection fails rather than crossing node boundaries.

Q12. Answer: B. Most Ingress Controllers (such as NGINX Ingress) do not send traffic to the backend Service's ClusterIP; instead, they watch EndpointSlices directly and load-balance straight to Pod IPs (or even if they hit the ClusterIP, the source IP seen by kube-proxy is the Ingress Controller Pod's IP, not the end user's browser IP). Cookie-based affinity must be configured at the Ingress/Gateway layer.

Q13. Answer: B. An Ingress object is just a declarative rule set. Without a running Ingress Controller watching the matching IngressClass (or a default IngressClass), nothing reconciles the Ingress or updates its status.loadBalancer.ingress (ADDRESS) field.

Q14. Answer: B. Kubernetes allows at most ONE IngressClass to carry ingressclass.kubernetes.io/is-default-class: "true". If two or more classes have this annotation set to "true", the IngressClass admission controller rejects any newly created Ingress that omits spec.ingressClassName.

Q15. Answer: C. With pathType: Exact, the URL path must match spec.rules[].http.paths[].path character-for-character with case sensitivity. Unlike Prefix, Exact on /api/v1 will NOT match /api/v1/ (trailing slash) or /api/v1/users.

Q16. Answer: A. For security isolation, a standard Kubernetes Ingress resource can only reference a TLS Secret located in the **same Namespace** as the Ingress itself. Because wildcard-tls was in default instead of production, NGINX Ingress fell back to its default self-signed "Fake Certificate".

Q17. Answer: B. When NGINX Ingress returns 503 Service Temporarily Unavailable, it means the Ingress rule matched the request, but the referenced Kubernetes Service either does not exist (typo in service name/port) or has zero healthy Pod endpoints in its EndpointSlice.

Q18. Answer: A. The built-in Kubernetes Ingress resource only models HTTP and HTTPS routing rules. Exposing arbitrary TCP/UDP services requires either a LoadBalancer/NodePort Service, custom ConfigMaps on specific controllers, or the modern **Gateway API** (TCPRoute/UDPRoute/TLSRoute).

Q19. Answer: B. Each Ingress Controller binary is started with a specific controller string (e.g., --controller-class=k8s.io/ingress-nginx-internal). It watches IngressClass resources whose spec.controller matches that string, and then reconciles any Ingress whose spec.ingressClassName matches that IngressClass's metadata.name.

Q20. Answer: B. spec.defaultBackend in an Ingress acts as a catch-all fallback target for any request arriving at the Ingress Controller that does not match any of the defined host or path rules.

Q21. Answer: B. Gateway API's role-oriented design assigns GatewayClass to the **Infrastructure Provider** (defining available load-balancer templates), Gateway to the **Cluster Operator** (defining listeners, ports, and TLS entry points), and HTTPRoute/GRPCRoute to **Application Developers** (defining application-specific routing rules).

Q22. Answer: A. In Gateway API, cross-namespace references (such as an HTTPRoute in storefront forwarding to a Service in security, or a Gateway referencing a TLS Secret in another Namespace) are denied by default (RefNotPermitted) unless the owner of the target Namespace (security) creates a ReferenceGrant explicitly permitting the source Namespace and kind.

Q23. Answer: A. For multi-tenant security, a Gateway listener defaults to allowedRoutes.namespaces.from: Same. For an HTTPRoute in the billing Names-

pace to attach to a Gateway in infra-gw, the Cluster Operator must update the listener's allowedRoutes.namespaces.from to All or Selector.

Q24. Answer: B. Gateway API supports weighted Canary traffic splitting natively in the core spec: an HTTPRoute rule can list multiple backendRefs, each with a weight field (e.g., 95 and 5), eliminating the need for vendor-specific Ingress annotations.

Q25. Answer: A. Just like Ingress, Gateway API resources require an active Gateway API controller (such as Cilium, Istio, Envoy Gateway, NGINX Gateway Fabric, or GKE Gateway Controller) running in the cluster to watch the GatewayClass and provision the underlying data plane.

Q26. Answer: A. Standard HTTPRoute rules include built-in filters (RequestHeaderModifier, ResponseHeaderModifier, RequestMirror, RequestRedirect, URLRewrite) and weighted backendRefs as portable, first-class API fields.

Q27. Answer: B. When both a Gateway listener and an attached HTTPRoute specify hostnames, traffic is only routed for the **intersection** of both sets. Since api.shop.example.com matches *.shop.example.com and admin.otherdomain.org does not, only api.shop.example.com is programmed.

Q28. Answer: B. This is the most famous Kubernetes security exam trap! In NetworkPolicy, placing namespaceSelector and podSelector under a single array item (one -) creates a logical **AND**. Adding a second hyphen (-) makes them two separate array elements evaluated as logical **OR**: rule 1 allows ALL Pods in env: prod, and rule 2 allows any Pod with role: backend in the local Namespace!

Q29. Answer: A. Once a Pod is selected by a NetworkPolicy with policyTypes: ["Egress"], all outbound traffic not explicitly whitelisted is dropped—including UDP/TCP port 53 traffic to CoreDNS! Always include an egress rule allowing port 53 (UDP and TCP) when restricting Pod egress.

Q30. Answer: B. The Kubernetes API server validates and stores NetworkPolicy objects regardless of the CNI plugin in use. However, actual packet filtering is performed by the CNI plugin. If the CNI (like basic Flannel) lacks a NetworkPolicy controller, the rules have zero effect on network traffic.

Q31. Answer: B. Pod isolation in NetworkPolicy is directional. Selecting a Pod with policyTypes: ["Ingress"] isolates only incoming traffic to that Pod; unless Egress is also listed in policyTypes (or an egress field is specified), outbound traffic from that Pod remains completely non-isolated.

Q32. Answer: B. An empty podSelector: {} matches **every Pod in that Namespace**. Combined with policyTypes: ["Ingress"] and no ingress allow list, it acts as a best-practice baseline "Default Deny All Ingress" policy for the finance Namespace.

Q33. Answer: A. With the legacy Endpoints API, all endpoints for a Service lived in a single API object. For a 2,500-Pod Service on a 500-node cluster, changing 1 Pod forced the control plane to send a massive payload containing all 2,500 IPs to all 500 nodes ($2,500 \times 500$ data amplification). EndpointSlices solved this by sharding endpoints into small slices.

Q34. Answer: B. By default, the EndpointSlice controller packs up to **100 endpoints** into each EndpointSlice (configurable up to 1,000). A Service with 350 Pods will therefore be represented by 4 EndpointSlice objects.

Q35. Answer: A. Historically, ready was always false as soon as a Pod started terminating, which caused dropped requests if replacement Pods weren't ready yet. EndpointSlice introduced serving (which reflects readiness regardless of termination state) and terminating, enabling kube-proxy and Gateways to still route to terminating-but-healthy Pods as a fallback when zero non-terminating Ready Pods exist.

Q36. Answer: B. volumeBindingMode: WaitForFirstConsumer intentionally delays dynamic PV creation and binding until a Pod referencing the PVC is scheduled onto a specific Worker Node. Once a Pod is created, the scheduler chooses a node and the volume is provisioned in the matching topology zone.

Q37. Answer: A. Because the StorageClass used volumeBindingMode: Immediate, the AWS EBS disk was created in us-east-1a before Kubernetes knew where the Pod would run. AWS EBS volumes are

zonal and cannot be attached across Availability Zones (1a → 1c), causing the scheduler to fail with `volume node affinity conflict`. Using `WaitForFirstConsumer` prevents this issue.

Q38. Answer: A. When a PVC is deleted and its PV has `reclaimPolicy: Retain`, the PV transitions to `Released`. It cannot bind to a new PVC because it still retains the old PVC's `spec.claimRef` metadata (and potentially sensitive data). To allow a new PVC to bind to that same PV with its existing data intact, an administrator must edit the PV and delete the `spec.claimRef` section, which transitions the PV back to `Available`.

Q39. Answer: B. A very common misconception is thinking `ReadWriteOnce` (`RWO`) means "one Pod". In Kubernetes, `ReadWriteOnce` means the volume can be mounted as read-write by a **single Node** (multiple Pods on that same node can mount it simultaneously). If you want to restrict access to a single Pod across the entire cluster, Kubernetes v1.29+ GA provides `ReadWriteOncePod` (`RWOP`).

Q40. Answer: A. Cloud block storage (like AWS EBS, GCP Persistent Disk, Azure Managed Disk) is typically `RWO` and can only attach to one Worker Node at a time. When a 3-replica Deployment schedules Pods across 3 different nodes, nodes 2 and 3 fail with `Multi-Attach error`. Shared multi-node access requires a `ReadWriteMany` (`RWX`) network filesystem like NFS, EFS, or CephFS.

Q41. Answer: B. PV-to-PVC binding is strictly a **1-to-1 exclusive mapping**. If no exact size match exists, the control plane binds the `10Gi` PVC to the smallest available PV that satisfies the request (`50Gi`). The entire `50Gi` PV is now exclusively bound to that single PVC.

Q42. Answer: A. Omitting `storageClassName` entirely is NOT the same as setting `storageClassName: ""`. When omitted, the `DefaultStorageClass` admission plugin automatically injects the cluster's default `StorageClass` name into the PVC, triggering dynamic provisioning. To bind to a static PV with no class, you must explicitly specify `storageClassName: ""` (empty string).

Q43. Answer: A. The built-in Storage Object in Use Protection controller adds the `kubernetes.io/pvc-protection` finalizer to PVCs (and `pv-protection` to PVs). If you delete a PVC while a Pod is still actively using it, the PVC enters `Terminating` and waits until the Pod is terminated before removing the finalizer and completing deletion.

Q44. Answer: A. To expand a PVC by increasing `spec.resources.requests.storage`, the associated `StorageClass` must have `allowVolumeExpansion: true` configured, and the underlying CSI driver must support volume expansion.

Q45. Answer: A. Volume expansion happens in two phases: first, the CSI Controller expands the physical cloud disk; second, the CSI Node plugin/`kubelet` expands the filesystem (`resize2fs` or `xfs_growfs`). `FileSystemResizePending` indicates phase 1 succeeded and phase 2 is waiting for a Pod to mount the volume (or restart if required).

Q46. Answer: A. Kubernetes never permits decreasing (shrinking) `spec.resources.requests.storage` on an existing bound PVC because shrinking block devices in-place risks corrupting the filesystem and losing data. To downsize, you must create a new smaller PVC and migrate the data at the application/file level.

Q47. Answer: A. While manually created static PVs default to `persistentVolumeReclaimPolicy: Retain`, PVs that are **dynamically provisioned** by a `StorageClass` default to `reclaimPolicy: Delete` unless the `StorageClass` explicitly sets `reclaimPolicy: Retain`.

Q48. Answer: A. When a PV/PVC uses `volumeMode: Block`, Kubernetes does not create a filesystem on the disk. Therefore, a Pod cannot mount it at a directory path using `volumeMounts`; it must map the raw block device into the container using `containers[].volumeDevices` and `devicePath`.

Q49. Answer: B. By design, scaling down or deleting a `StatefulSet` does **not** automatically delete the PVCs created by `volumeClaimTemplates` (unless the optional `persistentVolumeClaimRetentionPolicy` feature is explicitly configured to `Delete`). This safeguards stateful database volumes from accidental deletion during scale-down operations.

Q50. Answer: A. Setting `spec.securityContext.fsGroup: 10001` instructs `kubelet` to recursively `chown` and set the setgid bit on the mounted volume's files to GID `10001` before the container starts, allowing the non-root container process to write to the volume cleanly.